BABEŞ-BOLYAI UNIVERSITY, CLUJ-NAPOCA, ROMÂNIA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Compliance-as-Code AI Auditor

Checking Applications Against Policies and Regulations (e.g., EU AI Act)

– Advanced Machine Learning report –

Authors

Bâlc Felicia, **[TODO: specialisation]**, **[TODO: group]**, **[TODO: email]**
Drăghici Eduard, **[TODO: specialisation]**, **[TODO: group]**, **[TODO: email]**
Gog Raluca, **[TODO: specialisation]**, **[TODO: group]**, **[TODO: email]**

2026-2027

Abstract

Relevance. The EU Artificial Intelligence Act [7] turns ethical principles into legally binding, risk-tiered obligations (risk management, record-keeping, transparency, human oversight, robustness), which come on top of GDPR [6] and internal responsible-AI policies. Checking an application against these texts is today a manual, document-heavy activity that is repeated whenever the code or the regulation changes.

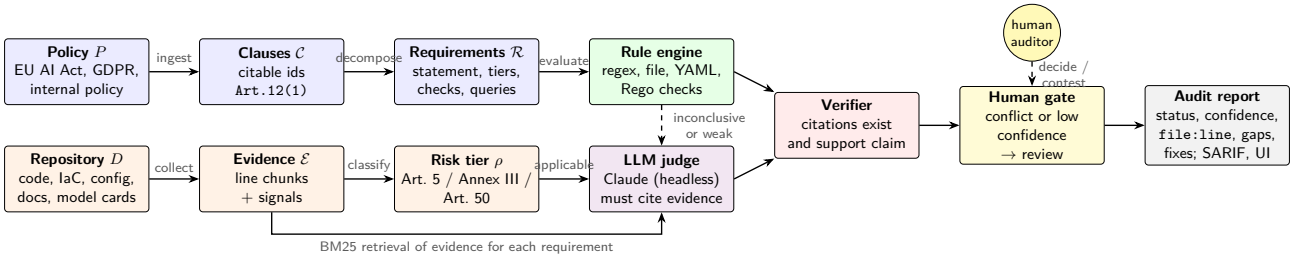
Method. We build a *Compliance-as-Code AI Auditor*: policies are parsed into citable clauses and decomposed into atomic, testable requirements (curated YAML or an LLM *PolicyAnalyst* agent); an application repository is turned into an index of evidence chunks (code, configuration, infrastructure-as-code, documentation, model cards, logs). Each requirement is evaluated by a *hybrid* orchestrator: a deterministic rule engine (regex, file, YAML-path and Rego checks) runs first; inconclusive or weak cases are sent to an LLM *ComplianceJudge* (Claude, invoked through the Claude Code CLI in headless mode) that receives BM25-retrieved evidence and must cite it; a *Verifier* removes hallucinated citations and refuses "compliant" claims without evidence; disagreements and low-confidence verdicts are escalated to a human reviewer. The output is an auditable report linking every requirement to `file:line` evidence or to a gap, with a risk tier, a scorecard, remediation advice and SARIF output for CI.

Data. Three policy corpora (selected EU AI Act provisions, GDPR, a fictional internal policy; 45 curated requirements) and a benchmark of five hand-labelled AI application repositories (225 requirement-level labels) designed with deliberate traps for rule-based checks (indirect evidence, decoys, requirements without rules).

Results.

RESULTS PENDING: one or two sentences with the headline numbers: accuracy / macro-F1 / false-compliance rate of deterministic vs LLM vs hybrid, share of requirements routed to the LLM, citation validity, cost and latency per audit.

Graphical abstract.



Contents

1 Project and team **1**
1.1 Project . . . 1
1.2 Team members, skills and roles . . . 1
1.3 Performed tasks and timeline . . . 1

2 Introduction **3**
2.1 What? Why? How? . . . 3
2.2 Report structure and original contributions . . . 3

3 Scientific problem **5**
3.1 Informal description . . . 5
3.2 Formal description . . . 6

4 State of the art and related work **8**
4.1 Policy-as-code and static compliance checking . . . 8
4.2 Requirements extraction and compliance checking with NLP . . . 8
4.3 Legal reasoning with LLMs . . . 8
4.4 Retrieval-augmented generation and LLM-as-a-judge . . . 9
4.5 Agentic orchestration . . . 9
4.6 Hallucination and citation verification . . . 9
4.7 AI Act scholarship and AI auditing . . . 9

5 Developed approach **11**
5.1 Overview . . . 11
5.2 Policy ingestion and decomposition . . . 12
5.3 Evidence collection and retrieval . . . 12
5.4 Risk-tier classification . . . 12
5.5 Deterministic rule engine . . . 13
5.6 LLM judge . . . 13
5.7 Hybrid orchestration . . . 13
5.8 Verifier: anti-hallucination contract . . . 13
5.9 Human review, scoring and reporting . . . 15
5.10 LLM backend, caching and CI . . . 15
5.11 Web dashboard . . . 16
5.12 Worked example . . . 16

6 Experiments **17**
6.1 Research questions and hypotheses . . . 17
6.2 Data: a hand-labelled benchmark . . . 17
6.2.1 Exploratory data analysis . . . 19

6.3 Protocol . . . 20
6.4 Results . . . 20
6.5 Complexity and sustainability . . . 22
6.6 Explainability . . . 22
6.7 Discussion . . . 22

7 Governance aspects **24**
7.1 Ethics . . . 24
7.2 Fairness . . . 24
7.3 Agency: what the system may decide and how far it may act . . . 25
7.4 Architecture: components required by that permission . . . 25
7.5 Accountability: who answers, what the record shows, how to contest . . . 25

8 SWOT analysis **27**

9 Conclusions and future work **28**

List of Tables

1.1 Team members, skills and roles. 1

4.1 Comparison with related approaches (✓ = supported, ∼ = partially). 10

5.1 Policy corpus and curated requirements. 12

6.1 Benchmark applications. 18
6.2 Ground-truth label distribution (45 requirements × 5 apps). 18
6.3 Main results on the 165 applicable labels (percentages; cost and latency per app audit). 21

7.1 Agency of the auditor: decisions it may take alone, with a human, or never. 25

List of Figures

1.1 Gantt diagram of the project (semester weeks; initials: FB = Bâlc Felicia, ED = Drăghici Eduard, RG = Gog Raluca). 2

3.1 Graphical abstract: policies become requirements, repositories become evidence; a hybrid rule/LLM evaluation with a citation verifier and a human-in-the-loop gate produces an auditable report. 6

5.1 Agents of the auditor. Every LLM agent is a single structured (JSON-schema) call without tool access; the orchestrator is ordinary code, so the control flow is fixed and auditable. 11

6.1 Policy corpus: clauses per article and clause length (left); requirements per category, severity and number of deterministic checks (right). 19
6.2 Benchmark repositories: files and evidence chunks per evidence kind. 19
6.3 Ground-truth labels per application (left) and applicable labels per category (right). . 19
6.4 Share of evidence chunks carrying each collector signal, per application. 20
6.5 Quality metrics per configuration. 21
6.6 Confusion matrices (rows: ground truth, columns: prediction) for hybrid (left) and LLM-only (right). 21
6.7 Cost–quality trade-off of the configurations. 21
6.8 Accuracy per requirement category. 21

8.1 SWOT analysis of the Compliance-as-Code AI Auditor. 27

List of Algorithms

1 Hybrid audit orchestration (`pipeline.run_audit`, mode = hybrid) 14
2 Verifier (`agents/verifier.py`) 15

Chapter 1

Project and team

1.1 Project

Title: Compliance-as-Code AI Auditor: Checking Applications Against Policies and Regulations (e.g., EU AI Act). **Course:** Advanced Machine Learning, Faculty of Mathematics and Computer Science, Babeş-Bolyai University, 2026-2027 (topic #5). **Repository layout:** `app/backend` (Python package `ccauditor`: engine, agents, CLI, REST API), `app/web` (Next.js dashboard), `app/benchmark` (labelled target applications), `docs/` (design and this report).

1.2 Team members, skills and roles

Table 1.1 identifies the team. **[TODO: confirm the role split and skills below; they are a proposal derived from the module boundaries of the code base.]**

Table 1.1: Team members, skills and roles.

Member	Identification	Skills and role
Bâlc Felicia	**[TODO: specialisation]**, **[TODO: group]** **[TODO: email]**	*Policy & governance lead.* Regulatory analysis (EU AI Act, GDPR), policy corpus in citable markdown, curated requirements YAML (statements, applicability, checks), benchmark labelling protocol, governance and ethics chapters.
Drăghici Eduard	**[TODO: specialisation]**, **[TODO: group]** **[TODO: email]**	*Architecture & LLM lead.* Data contracts, orchestrator (deterministic / LLM / hybrid), Claude Code headless backend with caching, agents (PolicyAnalyst, RiskClassifier, ComplianceJudge, Verifier, Remediator), CI integration.
Gog Raluca	**[TODO: specialisation]**, **[TODO: group]** **[TODO: email]**	*Evidence, UI & evaluation lead.* Evidence collectors and BM25 retriever, web dashboard (traceability, review queue), benchmark applications, experiment harness, metrics and figures.

1.3 Performed tasks and timeline

All members took part in the literature review, the weekly design reviews, the report and the presentation. Figure 1.1 shows the task plan over the 14 weeks of the semester. **[TODO: adjust the bars**

to the actual dates before submission.]

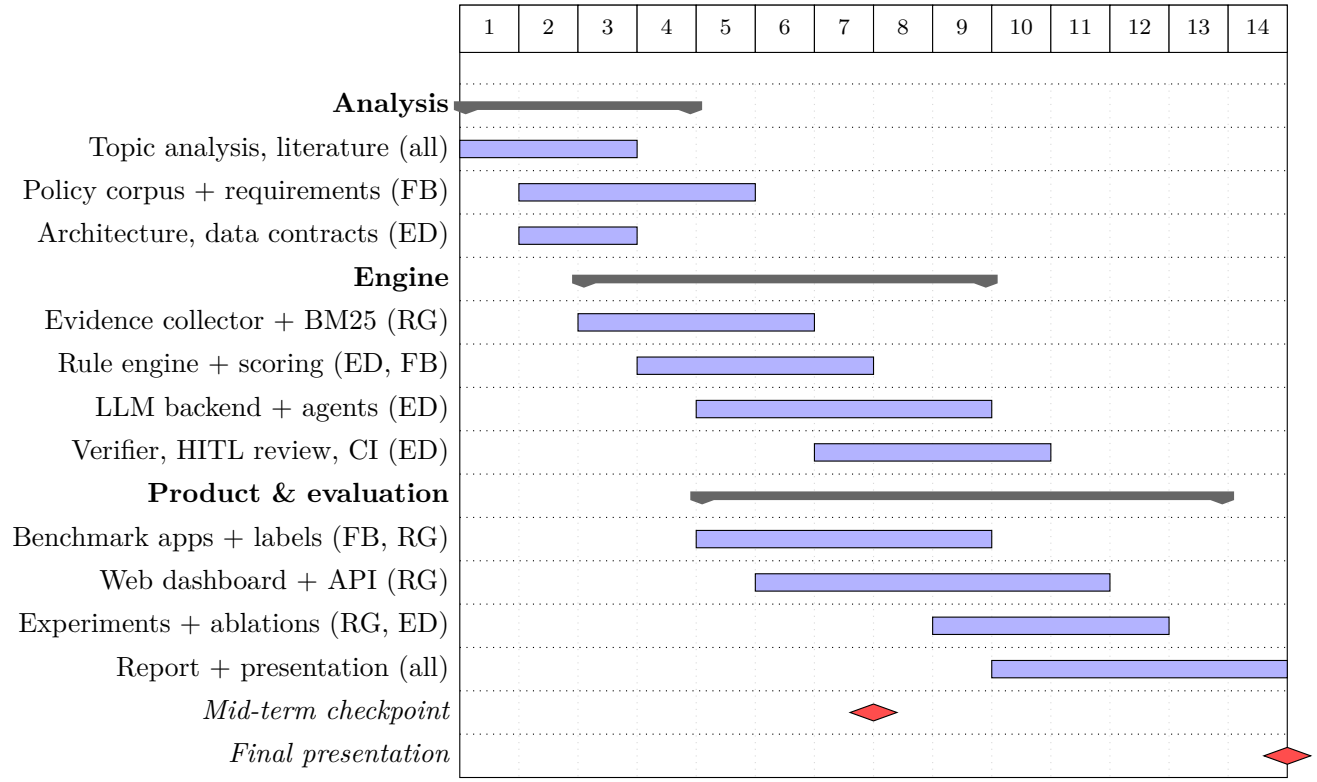


Figure 1.1: Gantt diagram of the project (semester weeks; initials: FB = Bâlc Felicia, ED = Drăghici Eduard, RG = Gog Raluca).

Chapter 2

Introduction

2.1 What? Why? How?

What. Given a *policy* (a regulation such as the EU AI Act [7] or GDPR [6], or an internal responsible-AI policy) and the *source repository* of an AI application, we want to know, for every obligation in the policy, whether the application satisfies it, and to be able to *prove* the answer: which file and which lines are the evidence, what is missing, and how to fix it. This is the idea of *compliance-as-code*: obligations become machine-checkable requirements that run continuously, like unit tests or the policy rules of the Open Policy Agent [25], instead of a yearly PDF audit.

Why. The AI Act is risk-tiered: prohibited practices (Art. 5), high-risk systems (Art. 6 and Annex III, e.g. recruitment and credit scoring) with heavy obligations (risk management, data governance, technical documentation, automatic logging, transparency to deployers, human oversight, accuracy and cybersecurity, Arts. 9–15), and lighter transparency duties for e.g. chatbots (Art. 50) [11, 32]. These obligations are written in natural language, are open-textured ("appropriate", "proportionate") and map to evidence that is scattered over code, infrastructure, configuration and documents. Manual auditing is slow, expensive and goes stale with every commit; purely rule-based scanners are fast but shallow (a regex cannot tell a commented-out logger from a real one); a large language model (LLM) can read and reason, but it hallucinates and is not, on its own, an acceptable basis for a legal conclusion [13].

How. We combine the three: (i) a *deterministic rule engine* gives cheap, reproducible verdicts where a requirement has a precise technical signature; (ii) an *LLM judge* with retrieval-augmented generation (RAG) [15] handles the open-textured requirements and the cases where rules are inconclusive; (iii) a *Verifier* and a *human-in-the-loop* gate make the result trustworthy: no "compliant" without a real citation, and every disagreement or low-confidence verdict goes to a person. The system is evaluated on a benchmark of five hand-labelled AI applications against three policies.

2.2 Report structure and original contributions

The main contributions are:

1. a **formalisation** of policy-to-repository compliance checking as grounded, selective multi-class prediction with citations (Chapter 3);
2. a **hybrid agentic auditor**: rule engine → LLM judge → verifier → human gate, with explicit routing rules, an anti-hallucination contract and full traceability (Chapter 5);
3. a **benchmark** of five realistic AI repositories with 225 requirement-level labels and deliberate traps for both rules and LLMs, plus an evaluation protocol covering quality, cost and latency (Chapter 6);
4. a **usable tool**: CLI, REST API, web dashboard with a review queue, SARIF output and CI regression gating, policy-version diffing;
5. a **governance analysis** of the auditor itself: what it may decide, which components that permission requires, and who is accountable (Chapter 7).

The report contains 35 bibliographical references and is structured as follows. Chapter 1 presents the team and the plan. Chapter 3 defines the scientific problem informally and formally. Chapter 4 reviews related work. Chapter 5 details the approach and Chapter 6 the experiments. Chapter 7 discusses ethics, fairness, agency, architecture and accountability, Chapter 8 gives a SWOT analysis and Chapter 9 concludes.

Chapter 3

Scientific problem

3.1 Informal description

An auditor receives two artefacts: a policy text and a software repository. The policy is long, hierarchical (articles, paragraphs, points) and mixes obligations with definitions and procedural provisions. The repository contains source code, infrastructure-as-code (Terraform, Kubernetes), configuration, documentation and possibly model cards and sample logs. The auditor must (1) find the obligations that are verifiable in a repository, (2) decide which of them apply to this system (this depends on the AI Act risk tier, which itself must be inferred from the intended purpose), (3) for each applicable obligation, locate evidence and decide whether it is met fully, partially or not at all, and (4) explain the decision with citations so that a third party can check it. The answer "I cannot tell, a human must look" is legitimate and preferable to a wrong answer.

Why an intelligent method. The mapping from legal text to code is not syntactic: Art. 12 "automatic recording of events" may be implemented as `logger.info(...)`, as a custom `DecisionJournal.record(...)` writing to object storage, or not at all despite a README claiming it. Rules alone suffer from false negatives (indirect evidence) and false positives (decoys such as commented code, TODOs, marketing claims). Reading and weighing heterogeneous evidence against an open-textured statement is a natural-language inference task, for which LLMs are the current best tool; their weaknesses (hallucination, sensitivity to context, cost) are what the rest of the design compensates for.

Advantages and disadvantages of the candidate methods.

- *Rule-based (policy-as-code)*: deterministic, explainable, cheap, CI-friendly; brittle, high maintenance, cannot judge sufficiency or proportionality.
- *LLM-only*: flexible, reads documents and code alike, writes good rationales; non-deterministic, may hallucinate evidence, costs money and time per call, sensitive to retrieval quality [16].
- *Hybrid with verification and abstention* (ours): spends LLM calls only where rules are weak, constrains the LLM to cited evidence and abstains when unsure; more moving parts, routing thresholds must be tuned.

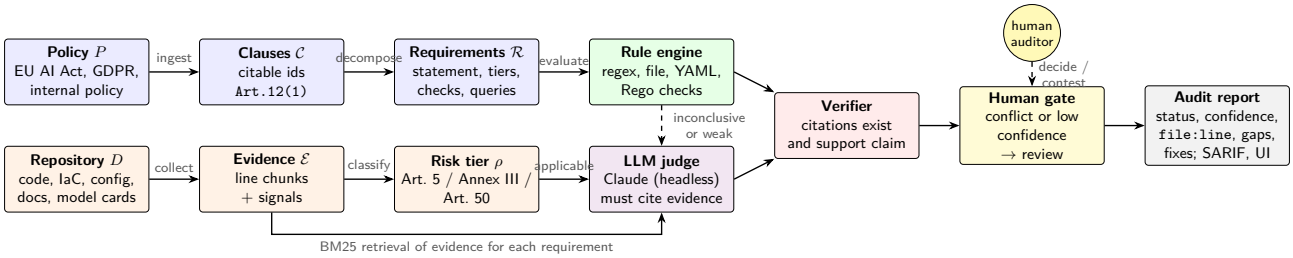


Figure 3.1: Graphical abstract: policies become requirements, repositories become evidence; a hybrid rule/LLM evaluation with a citation verifier and a human-in-the-loop gate produces an auditable report.

3.2 Formal description

Policy side. A policy P is a finite tree of clauses $\mathcal{C}(P) = \{c_1, \ldots, c_m\}$; each clause has a stable identifier (e.g. `eu-ai-act/Art.12(1)`), a text and a parent. A *decomposition* $\delta : \mathcal{C}(P) \to 2^{\mathcal{R}}$ produces requirements

$$r = (\text{id},\ C_r \subseteq \mathcal{C}(P),\ \kappa_r,\ s_r,\ A_r \subseteq \mathcal{L},\ w_r,\ K_r,\ Q_r,\ g_r),$$

where κ_r is a category (record keeping, transparency, human oversight, …), s_r a normative testable statement, A_r the risk tiers it applies to with $\mathcal{L} = \{\text{prohibited}, \text{high}, \text{limited}, \text{minimal}\}$, $w_r \in \{\text{critical}, \text{high}, \text{medium}, \text{low}\}$ a severity, K_r a (possibly empty) set of deterministic checks, Q_r retrieval queries and g_r guidance for the judge. δ is either curated by hand or computed by an LLM (PolicyAnalyst).

Repository side. A repository D is a set of text files. A collector φ maps it to an evidence set $\mathcal{E} = \varphi(D)$; each $e \in \mathcal{E}$ is a tuple $(\text{id}, k_e, \text{path}, [\ell_1, \ell_2], x_e, \sigma_e)$ with a kind $k_e \in \{\text{code}, \text{config}, \text{iac}, \text{doc}, \text{model_card}, \text{log}\}$, a line span, a snippet x_e and a set of regex signals σ_e (logging call, AI disclosure, human override, retention, …). A risk classifier $\rho : 2^{\mathcal{E}} \to \mathcal{L} \cup \{\text{unknown}\}$ assigns the AI Act tier.

Audit function. Let $\mathcal{S} = \{\text{C}, \text{P}, \text{NC}, \text{NR}, \text{NA}\}$ (compliant, partial, non-compliant, needs review, not applicable). The auditor is a function

$$f :\ \mathcal{R} \times 2^{\mathcal{E}} \ \longrightarrow\ \mathcal{S} \times [0,1] \times 2^{\mathcal{E}}, \qquad f(r, \mathcal{E}) = (\hat{y}_r,\ \pi_r,\ \hat{E}_r),$$

returning a status, a confidence and a set of cited evidence, subject to three hard constraints:

$$\text{(applicability)} \qquad \rho(\mathcal{E}) \notin A_r \ \Rightarrow\ \hat{y}_r = \text{NA}, \tag{3.1}$$

$$\text{(grounding)} \qquad \hat{E}_r \subseteq \mathcal{E}_r \subseteq \mathcal{E}, \text{ where } \mathcal{E}_r \text{ is the evidence shown to the evaluator}, \tag{3.2}$$

$$\text{(no unsupported compliance)} \qquad \hat{y}_r \in \{\text{C}, \text{P}\} \ \Rightarrow\ \hat{E}_r \neq \emptyset \text{ and } \hat{E}_r \text{ supports } s_r. \tag{3.3}$$

The status NR is an *abstention* (selective prediction): it is never a ground-truth label, it transfers the decision to a human at a cost λ.

Objective. With human labels $y_r \in \mathcal{S} \setminus \{\text{NR}\}$, a misclassification cost matrix Λ (asymmetric: predicting C when the truth is NC or P – a *false compliance* – is the most expensive error, because it hides a legal gap) and a per-call LLM cost, the design goal is

$$\min_{f} \sum_{r} \Big[\Lambda(y_r, \hat{y}_r)\mathbb{1}[\hat{y}_r \neq \text{NR}] + \lambda\mathbb{1}[\hat{y}_r = \text{NR}]\Big] \; + \; \mu \cdot \text{Cost}_{\text{LLM}}(f) \quad \text{s.t. (3.1)–(3.3).}$$

We do not optimise this objective numerically (there is no training set large enough); it motivates the routing and abstention thresholds of Chapter 5 and the metrics below.

Metrics. Let $\mathcal{A} = \{r : y_r \neq \text{NA}\}$ be the applicable requirements and $G = \{r \in \mathcal{A} : y_r \in \{\text{NC}, \text{P}\}\}$ the true *gaps*. Let $F = \{\text{NC}, \text{P}, \text{NR}\}$ be the predictions that *flag* a requirement for attention. The metrics (implemented in `app/benchmark/evaluate.py`) are:

- **Accuracy** $= \frac{1}{|\mathcal{A}|}\sum_{r\in\mathcal{A}}\mathbb{1}[\hat{y}_r = y_r]$ on the applicable labels; an NR or a wrong NA prediction counts as an error.
- **Macro-F1** over the graded classes $\{\text{C}, \text{P}, \text{NC}\}$ on $\mathcal{A}$.
- **Gap detection** (binary): gap precision $= |\{r \in G : \hat{y}_r \in F\}|/|\{r \in \mathcal{A} : \hat{y}_r \in F\}|$, gap recall $= |\{r \in G : \hat{y}_r \in F\}|/|G|$ and their F1.
- **False-compliance rate** FCR $= |\{r \in G : \hat{y}_r = \text{C}\}|/|G|$ (lower is better; the safety-critical failure of an auditor).
- **Needs-review rate** $= |\{r \in \mathcal{A} : \hat{y}_r = \text{NR}\}|/|\mathcal{A}|$ (human workload) and **accuracy@deferral**, the accuracy on the applicable items that were not deferred (selective accuracy).
- **Applicability accuracy** $= \frac{1}{|\mathcal{R}|}\sum_{r}\mathbb{1}\big[(\hat{y}_r = \text{NA}) = (y_r = \text{NA})\big]$ over all labels, and **risk-tier accuracy** of ρ over the applications.
- **Citation validity**: share of cited evidence ids that exist in the evidence index, and share of C verdicts carrying at least one citation [8, 28].
- **Complexity**: wall-clock latency per audit and per stage, number of LLM calls, input/output tokens, equivalent USD cost reported by the CLI, cache hit rate.

Computational complexity. With $n = |\mathcal{E}|$ chunks, $|\mathcal{R}|$ requirements and q queries per requirement, evidence collection is linear in the repository size, BM25 indexing is $O(n\bar{\ell})$ and each query $O(n)$; the rule engine is linear in the scanned text per check. The dominant cost is the number of LLM calls: $|\mathcal{R}_{\mathcal{A}}|$ judge calls in LLM mode versus $|\{r : \text{route}(r) = \text{LLM}\}|$ in hybrid mode (plus one risk call, and optionally one verifier call per positive verdict), each with a bounded prompt of at most 14 evidence items / 12 000 characters.

Chapter 4

State of the art and related work

We group related work by the sub-problem it addresses; for each group we state what they solve, how our setting differs and what we reuse. Table 4.1 summarises the comparison.

4.1 Policy-as-code and static compliance checking

Policy engines such as the Open Policy Agent [25] express organisational rules in a declarative language (Rego) and evaluate them against structured inputs (Kubernetes manifests, Terraform plans, API requests); static analysers report findings in the SARIF interchange format [24]. They are deterministic and CI-native, but presuppose that a human has already translated the regulation into precise rules, and they operate on structured data, not on prose documentation. Our rule engine is in this tradition (it even evaluates optional Rego checks) and we emit SARIF, but we add the translation step (requirements derived from clauses) and an LLM for the requirements that no rule can capture.

4.2 Requirements extraction and compliance checking with NLP

Regulatory requirements engineering has long studied how to turn legal text into software requirements: Breaux and Antón [4] systematically extract rights, obligations and constraints from HIPAA. Torre et al. [31] check the completeness of privacy policies against GDPR with NLP and ML, and Amaral Cejas et al. [2] check data processing agreements against GDPR with an NLP pipeline built on a manually derived set of "shall" requirements. Hassani et al. [12] argue that LLMs can replace much of this hand-crafted pipeline. These works check *documents against documents*; we check a *software repository* (code, IaC, configuration and documents) against a regulation, and we target the AI Act, which is newer and has no labelled corpus.

4.3 Legal reasoning with LLMs

LexGLUE [5] and LegalBench [10] benchmark language models on legal understanding and reasoning tasks (issue spotting, rule application, interpretation). They show that LLMs perform well on rule-application tasks but unevenly across task types, which supports using them as *judges of a single, well-scoped requirement* rather than as end-to-end auditors. Our benchmark is task-specific (requirement × repository) and much smaller, but its labels are grounded in repository evidence.

4.4 Retrieval-augmented generation and LLM-as-a-judge

RAG [15] conditions generation on retrieved passages; we use sparse BM25 retrieval [29], which suits code identifiers and needs no embedding model, and cap the context because long contexts degrade the use of mid-context information [16]. Zheng et al. [35] show that strong LLMs agree with human preferences at a level comparable to inter-human agreement, but exhibit position, verbosity and self-enhancement biases; we therefore give the judge a structured schema, a single requirement at a time and a hint from the rule engine that it is explicitly told may be wrong. Calibration of LLM self-reported confidence is imperfect but informative [14]; we use it only for routing to a human, not as a probability of legal correctness.

4.5 Agentic orchestration

ReAct [34] interleaves reasoning with tool calls; AutoGen [33] composes multiple conversing agents. Our agents are deliberately *not* free-running: the orchestrator is a fixed program, each agent is a single structured call without tools (the Claude Code CLI is run with tools disabled), which bounds cost and makes every step auditable.

4.6 Hallucination and citation verification

Hallucination is pervasive in natural language generation [13]. Attribution can be framed as whether generated statements are supported by identified sources (AIS) [28]; ALCE [8] measures citation recall and precision of LLM answers; FActScore [20] decomposes outputs into atomic facts checked against a knowledge source and SelfCheckGPT [18] detects hallucinations by sampling consistency. Our Verifier applies the same principle with a cheaper, stricter contract adapted to auditing: citations must refer to evidence ids actually shown to the judge, positive verdicts require at least one valid citation, and an optional second LLM pass checks support claim-by-claim.

4.7 AI Act scholarship and AI auditing

Veale and Zuiderveen Borgesius [32] analyse the structure and weaknesses of the AI Act proposal, including its reliance on self-assessment and harmonised standards; Hacker et al. [11] discuss obligations for generative models. Raji et al. [27] propose an end-to-end internal audit framework (SMACTR) whose artefacts our tool can help produce; Mökander et al. [22] propose layered (governance, model, application) audits of LLMs. Model cards [21] and datasheets [9] are documentation artefacts our evidence collector treats as a separate kind and our ArtifactGenerator can draft. We position the tool as *support for the internal, application-level audit*, not as a conformity assessment.

Table 4.1: Comparison with related approaches (✓ = supported, ∼ = partially).

Approach	**Legal text**	**Code/IaC**	**Open-textured**	**Citations**	**Abstains**	**CI**
OPA / Rego, SARIF tools [24, 25]	–	✓	–	✓	–	✓
NLP compliance of documents [2, 31]	✓	–	∼	∼	–	–
Legal LLM benchmarks [5, 10]	✓	–	✓	–	–	–
LLM + citations [8, 20]	–	–	✓	✓	–	–
Ours	✓	✓	✓	✓	✓	✓

Chapter 5

Developed approach

5.1 Overview

The auditor is a Python package (`ccauditor`, ≈4 000 lines) organised as a pipeline of stages that exchange typed data contracts (Pydantic models `Clause`, `Requirement`, `Evidence`, `Verdict`, `RiskAssessment`, `AuditReport`). The same pipeline is exposed through a CLI (`ccauditor audit|policies|diff-policy|ci|review`), a FastAPI REST service and a Next.js web dashboard. Figure 5.1 shows the agents and what each one is allowed to see and produce. Three evaluation modes share the pipeline: *deterministic* (rules only, no LLM call), *llm* (the judge evaluates every applicable requirement) and *hybrid* (rules first, LLM only where needed).

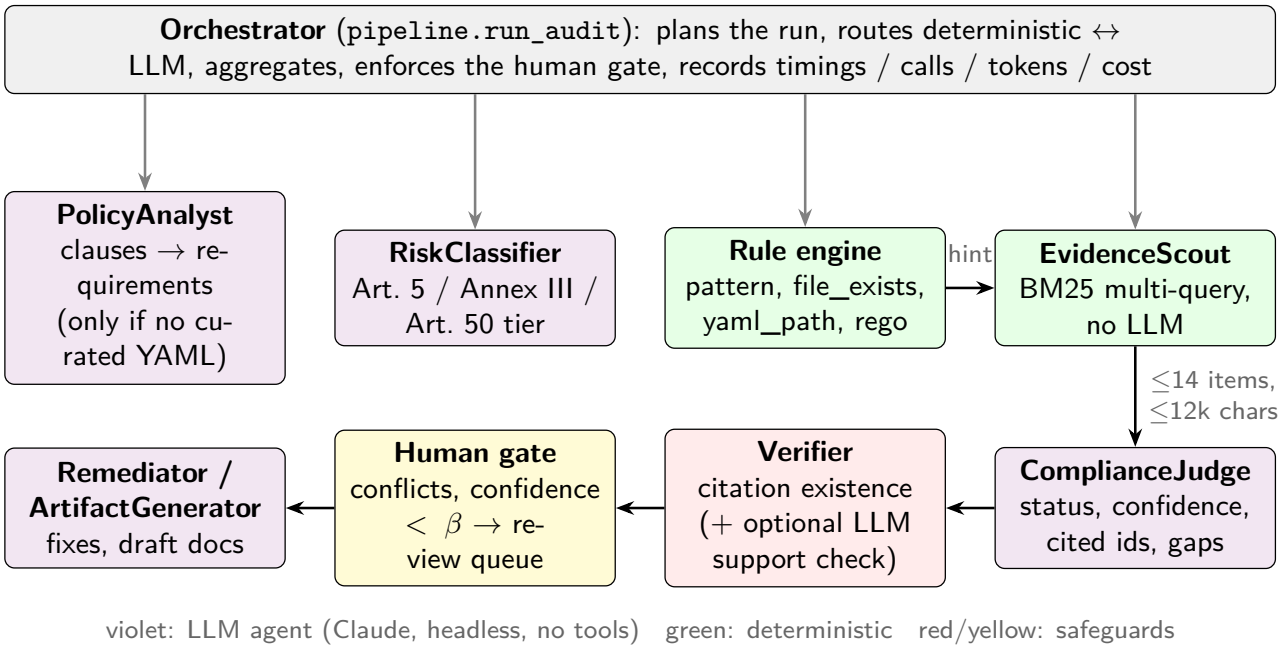


Figure 5.1: Agents of the auditor. Every LLM agent is a single structured (JSON-schema) call without tool access; the orchestrator is ordinary code, so the control flow is fixed and auditable.

5.2 Policy ingestion and decomposition

Policies are stored as markdown with a YAML front matter (`id`, `version`, `source`); headings "`## Art. 12 | Record-keeping`" and "`### 1`" are parsed into clauses with stable, citable ids (`eu-ai-act/Art.12`, `eu-ai-act/Art.12(1)`) and parent links. Requirements are *curated* in YAML (statement, clause ids, category, applicable risk tiers, severity, retrieval queries, judge guidance and deterministic checks). When no curated file exists for a policy, the **PolicyAnalyst** agent decomposes the clauses in batches, with the instruction to emit only atomic, repository-testable requirements, to use only the given clause ids and to skip definitional or authority-facing clauses. Table 5.1 summarises the bundled corpus. A **policy evolution** module diffs two versions of a policy at clause level (added / removed / modified, with parent–child propagation) and lists the requirements and existing reports impacted by the change.

Table 5.1: Policy corpus and curated requirements.

Policy	Clauses	Req.	with checks	checks	check types
EU AI Act (selected provisions)	108	23	22	25	19 pattern, 6 file_exists
GDPR (selected articles)	32	11	10	12	9 pattern, 3 file_exists
ACME RAI policy (fictional, v1)	30	11	10	12	11 pattern, 1 file_exists
Total	170	45	42	49	

5.3 Evidence collection and retrieval

The collector walks the repository (skipping binaries, vendored and build directories), classifies each file into one of six evidence kinds (code, config, IaC, doc, model card, log) and splits it into overlapping windows of 40 lines (overlap 10; files of at most 50 lines are a single chunk). Each chunk receives a stable id (hash of path and line span) and the set of *signals* raised by 16 lightweight regex detectors (logging call, AI disclosure, human override, retention configuration, PII field, model call, consent, encryption, access control, rate limit, data documentation, bias/fairness, accuracy metrics, risk management, monitoring, anonymisation).

Retrieval uses BM25 [29] over path + snippet with a code-aware tokenizer (camelCase and snake_case are split, the joined form is kept), boosted by 1.5× when a query term matches a chunk signal. The **EvidenceScout** builds several queries per requirement (curated queries, keywords of title + statement, one category-specific expansion), merges the hits with a reciprocal-rank bonus, *pins* the evidence already cited by the rule engine first, and caps the result at 14 items and 12 000 characters (1 500 per snippet). No embedding model is needed and the scout makes no LLM call.

5.4 Risk-tier classification

Applicability depends on the AI Act tier. A deterministic pre-classifier matches regex triggers for prohibited practices (Art. 5: social scoring, manipulation, workplace emotion recognition, untargeted facial scraping, predictive policing, real-time remote biometric identification), Annex III high-risk areas (biometrics, critical infrastructure, education, employment, credit scoring, …) and Art. 50 transparency

cases (chatbots, generative content, deep fakes) mostly in documentation, and returns the most severe tier with the triggering evidence. In LLM modes the **RiskClassifier** agent receives this result as a hint plus retrieved descriptions of the intended purpose, and returns tier, triggers and cited evidence. Requirements whose A_r excludes the tier get NA deterministically (Eq. 3.1); the judge is not allowed to output NA.

5.5 Deterministic rule engine

Each check returns *pass*, *fail* or *unknown*: `pattern` (a regex must be present / absent in files selected by glob or evidence kind), `file_exists`, `yaml_path` (a key exists or equals a value in YAML/JSON) and `rego` (evaluated with OPA [25] if installed, otherwise unknown). Results are aggregated per requirement: all resolved checks pass ⇒ C; none pass ⇒ NC; mixed ⇒ P; no resolved check ⇒ NR. The rule engine has a fixed prior confidence per status (0.9 C, 0.8 NC, 0.5 P, 0.2 NR), cites the matching chunks and lists the failed checks as gaps.

5.6 LLM judge

The **ComplianceJudge** receives one requirement (statement, guidance, the text of its source clauses), the scout's evidence rendered with short aliases (`[E1]`, `[E2]`, …, which are far less error-prone than hash ids), and, in hybrid mode, the rule-engine verdict explicitly labelled as a hint that "can be wrong (regexes are shallow)". The system prompt fixes the contract: decide only from the excerpts; cite only given ids; absence of evidence is not compliance; use NR for genuinely ambiguous evidence; report a calibrated confidence (use < 0.6 when unsure); list concrete gaps and an actionable remediation. The answer must follow a JSON schema (`status, confidence, rationale, evidence_ids, gaps, remediation`); aliases are mapped back to evidence ids, and unknown ids are kept verbatim so that the Verifier can remove them.

5.7 Hybrid orchestration

Algorithm 1 gives the orchestration. Two thresholds control the trade-off between cost and quality: the routing threshold $\tau = 0.75$ (a deterministic verdict with lower confidence is sent to the judge, which in practice routes every P and NR rule verdict) and the human-review threshold $\beta = 0.6$. Requirements with judge guidance but no checks are always routed. The COMBINE step never lets the LLM silently overrule the rules: a hard disagreement (C vs. NC) becomes NR with both rationales shown to the reviewer; agreement raises the confidence by 0.05.

5.8 Verifier: anti-hallucination contract

The Verifier (Algorithm 2) enforces Eqs. (3.2)–(3.3). Its deterministic pass runs on *every* verdict, including rule-engine ones: citations that do not belong to the evidence actually shown to the evaluator are removed and the confidence is multiplied by 0.8; a positive verdict (C or P) without any remaining valid citation is downgraded to NR with confidence at most 0.4 ("no hallucinated compliance"). An optional LLM pass, in the spirit of attribution checking [20, 28], shows a second, sceptical model only

Algorithm 1 Hybrid audit orchestration (`pipeline.run_audit`, mode = hybrid)

Require: repository D, policy ids, LLM backend B, thresholds $\tau = 0.75$, $\beta = 0.6$
Ensure: audit report with verdicts V, scorecard, statistics

$\mathcal{R} \leftarrow$ curated requirements, or $\textsc{PolicyAnalyst}_B$(clauses) if none
$\mathcal{E} \leftarrow \varphi(D)$; build BM25 index over $\mathcal{E}$
$t \leftarrow \textsc{RiskClassifier}_B(\mathcal{E}, \text{hint} = \textsc{HeuristicTier}(\mathcal{E}))$
$J \leftarrow \emptyset$ {requirements routed to the LLM judge}
for all $r \in \mathcal{R}$ **do**
  if $t \notin A_r$ **then**
    $V[r] \leftarrow (\textsc{na}, \text{conf}(t), \emptyset)$; **continue**
  end if
  $d_r \leftarrow \textsc{Verify}(\textsc{RuleEngine}(r, D, \mathcal{E}), \text{ids}(\mathcal{E}), \text{requireEvidence} = \text{false})$
  if $d_r.\text{status} \in \{\textsc{p}, \textsc{nr}\}$ **or** $d_r.\text{conf} < \tau$ **or** $(K_r = \emptyset$ **and** $g_r \neq \varepsilon)$ **then**
    $J \leftarrow J \cup \{r\}$
  else
    $V[r] \leftarrow d_r$ {confident rule verdict, no LLM call}
  end if
end for
for all $r \in J$ **in parallel** (bounded by the backend concurrency) **do**
  $\mathcal{E}_r \leftarrow \textsc{EvidenceScout}(r, \text{pinned} = d_r.\text{ids})$ {BM25, ≤ 14 items}
  $\ell_r \leftarrow \textsc{ComplianceJudge}_B(r, \mathcal{E}_r, \text{hint} = d_r)$
  $\ell_r \leftarrow \textsc{Verifier}(r, \ell_r, \mathcal{E}_r \cup d_r.\text{ids})$ {Algorithm 2}
  if $\{d_r.\text{status}, \ell_r.\text{status}\} = \{\textsc{c}, \textsc{nc}\}$ **then**
    $v \leftarrow (\textsc{nr}, \min(0.5, d_r.\text{conf}, \ell_r.\text{conf}))$, keep both rationales
  else if $d_r.\text{status} = \ell_r.\text{status}$ **then**
    $v \leftarrow \ell_r$ with conf $\leftarrow \min(1, \max(d_r.\text{conf}, \ell_r.\text{conf}) + 0.05)$
  else
    $v \leftarrow \ell_r$ {e.g. rule p, judge c}
  end if
  $v \leftarrow \textsc{Verify}(v, \mathcal{E}_r \cup d_r.\text{ids})$
  if $v.\text{status} \notin \{\textsc{nr}, \textsc{na}\}$ **and** $v.\text{conf} < \beta$ **then**
    $v.\text{status} \leftarrow \textsc{nr}$ {human-in-the-loop gate}
  end if
  $V[r] \leftarrow v$
end for
scorecard $\leftarrow \textsc{Score}(\mathcal{R}, V)$; record timings, calls, tokens, cost, cache hits
return report(t, V, cited evidence, scorecard, statistics)

the cited excerpts and asks whether they support the claim; unsupported claims become NR while the citations stay visible to the reviewer.

Algorithm 2 Verifier (`agents/verifier.py`)

Require: requirement r, verdict $v = (y, \pi, \hat{E})$, evidence set provided to the evaluator $\mathcal{E}_r$, flag *useLLM*
Ensure: verified verdict

CHECK:
  $\hat{E}_{\text{bad}} \leftarrow \hat{E} \setminus \mathcal{E}_r$; $\hat{E} \leftarrow \hat{E} \cap \mathcal{E}_r$
if $\hat{E}_{\text{bad}} \neq \emptyset$ **then**
  $\pi \leftarrow 0.8\,\pi$; append "removed $|\hat{E}_{\text{bad}}|$ citation(s)" to the rationale
end if
if *requireEvidence* **and** $y \in \{\text{C}, \text{P}\}$ **and** $\hat{E} = \emptyset$ **then**
  $y \leftarrow$ NR; $\pi \leftarrow \min(\pi, 0.4)$ {no compliance without evidence}
end if
mark v as verified
if *useLLM* **and** $y \in \{\text{C}, \text{P}\}$ **then**
  $(s, U) \leftarrow$ LLM("do excerpts $\hat{E}$ support status y for statement s_r?")
  $\hat{E} \leftarrow \hat{E} \setminus U$
  if $s =$ false **or** $\hat{E} = \emptyset$ **then**
    $y \leftarrow$ NR; $\pi \leftarrow \min(\pi, 0.5)$; keep original citations for the reviewer
  end if
end if
return $(y, \pi, \hat{E})$

5.9 Human review, scoring and reporting

Every verdict carries status, confidence, method (deterministic / llm / hybrid / human), a *verified* flag, rationale, gaps, remediation and the cited evidence with path and line span. Reviewers resolve NR items (or contest any other verdict) by submitting a decision with their name and a justification; the override is recorded in the rationale and in the report history, the method becomes `human` and the scorecard is recomputed. Category scores are $(\#\text{C} + 0.5\,\#\text{P})/\#\text{applicable} \times 100$; the overall score uses the same credit weighted by severity (critical 4, high 3, medium 2, low 1). Reports are rendered as JSON, Markdown and SARIF [24] (so that gaps appear as code-scanning alerts on the cited lines), and an `explain` function prints the full justification of a single verdict. The **ArtifactGenerator** drafts missing artefacts (technical documentation, risk assessment, transparency notice, model card [21]) from the findings, with explicit `[TODO]` placeholders instead of invented facts.

5.10 LLM backend, caching and CI

The LLM is Claude, called through the **Claude Code CLI in headless mode** [3] (`claude -p --output-format json --json-schema ...`) with the developer's subscription, so no API key is stored. Each call runs with tools, MCP servers, slash commands, settings and session persistence disabled, in an empty temporary working directory: the model can read only the prompt. Responses are parsed (with a fallback that extracts the first balanced JSON object), validated against the schema

and retried up to twice with the validation errors appended. A content-addressed **disk cache** (SHA-256 of model, system prompt, prompt and schema) makes re-audits of unchanged code free and deterministic; the backend records calls, cache hits, failures, input/output tokens, the equivalent USD cost reported by the CLI and latency. A semaphore bounds the concurrency (default 4). A `FakeBackend` makes the whole pipeline testable offline. In CI, `ccauditor ci` audits the current tree, compares it with a baseline report (per-requirement status rank NC < NR < P < C) and fails the build on regressions at or above a chosen severity.

5.11 Web dashboard

The Next.js dashboard (over the REST API: `/policies`, `/audit`, `/reports/compare`, `/reports/render`, `/reports/review`, `/policies/diff`) lets a user browse policies and clauses, launch an audit (mode, policies, model), and inspect a report in four tabs: *overview* (score gauge, per-category charts, risk tier and triggers), *requirements* (filterable table; a side sheet shows the statement, source clauses, rationale, cited code snippets, gaps and remediation), *review* (the human-in-the-loop queue with a decision form) and *traceability* (clause → requirement → evidence links).

RESULTS PENDING: add one or two dashboard screenshots (`figures/dashboard_*.png`).

5.12 Worked example

Consider requirement `AIA-REC-01` (Art. 12(1)–(2), automatic logging of every decision with timestamp, input reference, output and model version) on `hireright-screening`, a high-risk (Annex III 4(a)) CV-ranking service. The ground truth is C: every ranking is recorded by a custom `DecisionJournal.record(...)` on S3 Object Lock, not by a `logger` call.

1. The tier is *high* (recruitment trigger in the README and model card), so r applies.
2. The rule check is a regex for a `logger` / `audit_log` call that mentions a prediction, decision, score or output in code; it fails, so the rule engine says NC with confidence 0.8.
3. Since $0.8 \geq \tau$, Algorithm 1 keeps this confident but wrong rule verdict in hybrid mode, whereas in LLM mode the scout's queries "journal record decision" and "audit log decision" retrieve `journal.py` and the call sites in `api.py`, and the judge can cite them for C.

The example illustrates the central trade-off quantified in Chapter 6: confident rules save LLM calls but propagate rule false negatives, while the LLM recovers indirect evidence at a higher cost; routing confident NC rule verdicts to the judge (or lowering the NC prior) is one of the ablations.

RESULTS PENDING: replace step 3 with the actual verdicts of each mode for this item from the benchmark run.

Chapter 6

Experiments

6.1 Research questions and hypotheses

RQ1 *Quality.* Does the hybrid mode reach at least the accuracy and macro-F1 of the LLM-only mode, and exceed the deterministic mode, on applicable requirements?

RQ2 *Safety.* Which mode minimises the false-compliance rate, and how much human workload (needs-review rate) does that cost?

RQ3 *Cost.* How many LLM calls, tokens, dollars and seconds does each mode need per audit, and how much does the cache save on a re-audit?

RQ4 *Safeguards.* What do the Verifier, the LLM support check and the model size contribute (ablations)?

The *independent variables* are the mode, the model, and the safeguard switches; the *dependent variables* are the metrics of Section 3.2.

6.2 Data: a hand-labelled benchmark

There is no public dataset of application repositories labelled against the AI Act, so we built one (`app/benchmark`). Five small but realistic repositories cover the four risk tiers and different stacks (Table 6.1); each contains code, configuration, infrastructure-as-code, documentation, model cards and sample logs as real teams would write them. Every one of the 45 curated requirements was labelled for every app (225 labels, 165 applicable), with a justification and file references. Labels were written *before* running the rule engine, by reading the repository against the requirement statement; claims without backing (README marketing, TODOs, commented code) are not evidence; NA is used only when the requirement's risk tiers exclude the app's tier, so it is never a judgement call. Table 6.2 gives the label distribution.

The benchmark is deliberately adversarial for rules: *indirect evidence* (regex false negatives, e.g. a `DecisionJournal` instead of `logger`, "pseudonymous" instead of "pseudonymise"), *decoys* (regex false positives, e.g. `# logger.info(...)`, `override_allowed = False`, a TODO to label AI replies) and three requirements without any check. It is also adversarial for LLMs: plausible but unbacked

Table 6.1: Benchmark applications.

App	Domain	AI Act tier	Compliance profile
`hireright-screening`	CV ranking	high (Annex III 4(a))	mostly compliant; logging via a custom journal, kill switch with an unusual name; gaps: bias evaluation, ROPA, DPIA
`quickhire-ranker`	CV ranking MVP	high (Annex III 4(a))	mostly non-compliant: auto-rejection, protected attributes as features, no auth; decoys (commented logger, "fully GDPR compliant" README)
`supportbot-genai`	support chatbot	limited (Art. 50)	disclosure banner, erasure endpoint; gaps: unlabelled AI replies (TODO only), 2-year transcript retention
`credit-lens`	credit scoring	high (Annex III 5(b))	strong governance; gaps: 30-day log retention, automatic declines up to EUR 25 000, no instructions for use
`spamshield`	spam filter	minimal	most AI Act duties N/A; GDPR largely compliant; gaps: incident procedure, on-request erasure

Table 6.2: Ground-truth label distribution (45 requirements × 5 apps).

App	Tier	C	P	NC	NA	total
credit-lens	high	32	7	3	3	45
hireright-screening	high	37	1	4	3	45
quickhire-ranker	high	1	2	39	3	45
spamshield	minimal	15	2	1	27	45
supportbot-genai	limited	15	2	4	24	45
all		**100**	**14**	**51**	**60**	**225**

claims in READMEs and disproportionate-but-present controls (retention defined but too long) that require reading thresholds.

6.2.1 Exploratory data analysis

Figures 6.1–6.4 describe the inputs. The AI Act dominates the corpus, with long and structurally varied clauses (Annex III, Art. 9, Art. 13). Most requirements have exactly one deterministic check, and three have none (Figure 6.1). The applications differ in the mix of evidence kinds: `hireright` and `credit-lens` are documentation-heavy, `quickhire` is mostly code (Figure 6.2). The labels are imbalanced per application by design (Figure 6.3). The signal heat map (Figure 6.4) shows why a signal alone is weak evidence: for example, 50% of `quickhire` chunks carry a PII signal, yet that application has no data protection.

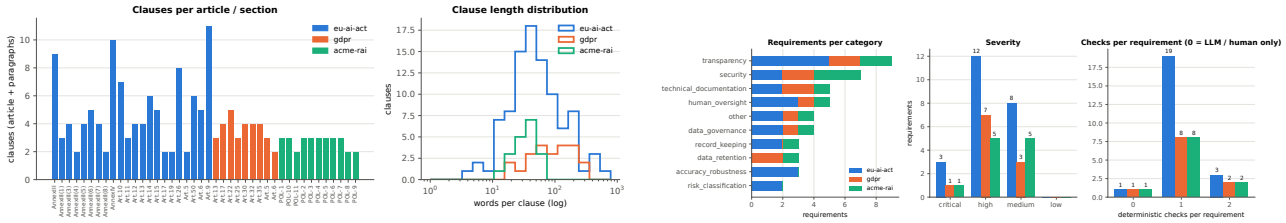


Figure 6.1: Policy corpus: clauses per article and clause length (left); requirements per category, severity and number of deterministic checks (right).

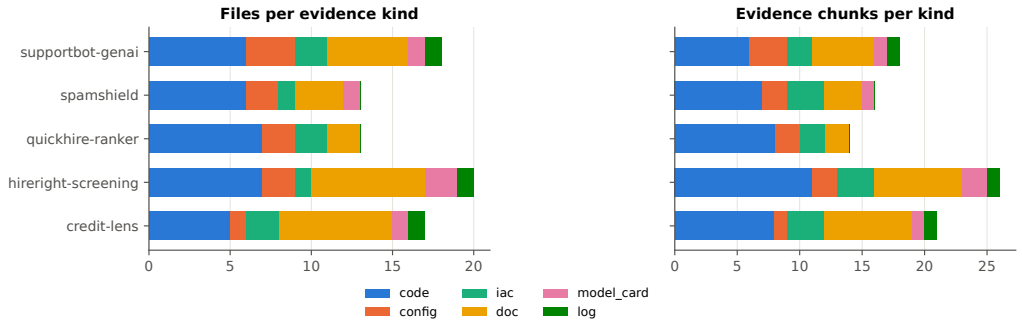


Figure 6.2: Benchmark repositories: files and evidence chunks per evidence kind.

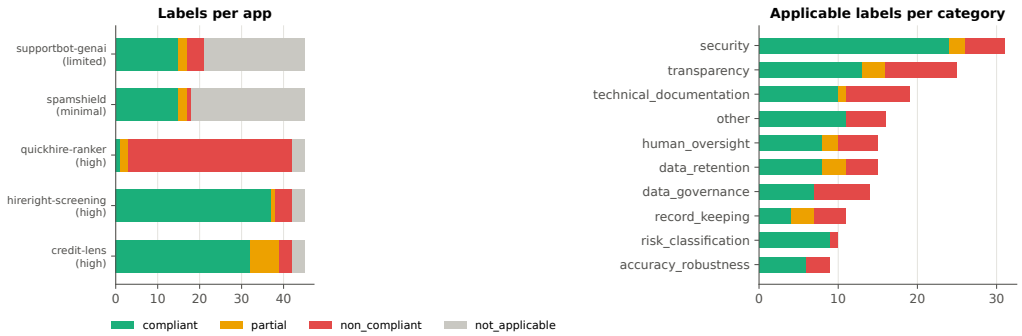


Figure 6.3: Ground-truth labels per application (left) and applicable labels per category (right).

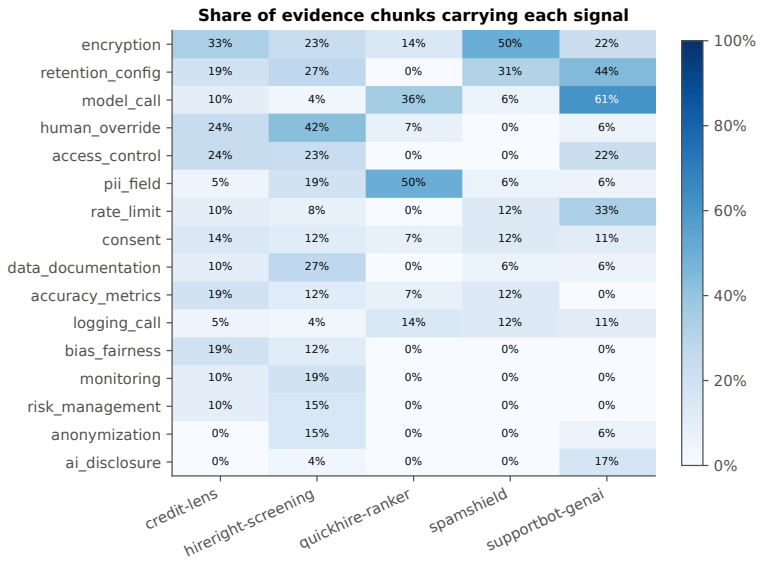


Figure 6.4: Share of evidence chunks carrying each collector signal, per application.

6.3 Protocol

- **Configurations.** *deterministic*; *hybrid* and *llm* with Claude Sonnet (default) and Claude Haiku (smaller, cheaper); all three policies at once on each app (45 requirements per audit).
- **Runs.** `app/benchmark/evaluate.py` audits the five apps in parallel processes (3 concurrent LLM calls per audit), stores every full report, the per-item predictions and a 5×5 confusion matrix per configuration. Temperature cannot be set through the CLI; variance is estimated by repeating cold (uncached) runs.

  RESULTS PENDING: number of repetitions and seeds.
- **Ablations.** (a) Verifier disabled (`--no-verifier`); (b) LLM claim-support check enabled (`--verify-llm`); (c) model size (Sonnet vs. Haiku); (d) routing threshold τ / routing of confident NC rule verdicts; (e) cache: cold run vs. warm re-audit of unchanged code.
- **Statistics.** 95% bootstrap confidence intervals over items (1 000 resamples) for accuracy, macro-F1 and FCR, and McNemar's exact test on paired per-item correctness between modes.
- **Hardware.** **[TODO: CPU / RAM / OS of the machine]**; LLM inference runs on Anthropic's servers via the Claude Code CLI **[TODO: CLI version]**, so local memory is dominated by the Python process.

6.4 Results

Table 6.3 is filled from `app/benchmark/results/SUMMARY.md`.

RESULTS PENDING: fill Table 6.3 (or provide `docs/report/results_tables.tex`, which replaces this block automatically), add bootstrap CIs and McNemar p-values.

Table 6.3: Main results on the 165 applicable labels (percentages; cost and latency per app audit).

Config	Acc	M-F1	Gap P	Gap R	FCR↓	NR	Acc@def	Cit. valid	LLM calls	Cost $ / s
deterministic	[]	[]	[]	[]	[]	[]	[]	[]	0	0 / []
hybrid (Sonnet)	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
llm (Sonnet)	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
hybrid (Haiku)	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
hybrid, no Verifier	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
hybrid, LLM verifier	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]

RESULTS PENDING: figure `figures/mode_comparison.{pdf,png}` not generated yet. Bar chart: accuracy, macro-F1, gap F1, FCR, needs-review per configuration.

Figure 6.5: Quality metrics per configuration.

RESULTS PENDING: figure `figures/confusion_hybrid.{pdf,png}` not generated yet. Confusion matrix (truth × prediction) of the hybrid mode.

RESULTS PENDING: figure `figures/confusion_llm.{pdf,png}` not generated yet. Confusion matrix of the LLM-only mode.

Figure 6.6: Confusion matrices (rows: ground truth, columns: prediction) for hybrid (left) and LLM-only (right).

RESULTS PENDING: figure `figures/cost_quality.{pdf,png}` not generated yet. Scatter: cost (USD) or LLM calls per audit vs. accuracy / FCR, one point per configuration.

Figure 6.7: Cost–quality trade-off of the configurations.

RESULTS PENDING: figure `figures/per_category.{pdf,png}` not generated yet. Heat map or grouped bars: accuracy per requirement category and mode.

Figure 6.8: Accuracy per requirement category.

6.5 Complexity and sustainability

Time and memory. The deterministic mode is CPU-only and runs in well under a second per application; memory is bounded by the evidence index (a few thousand chunks for our apps). In LLM modes wall-clock time is dominated by the judge calls and scales with the number of routed requirements divided by the concurrency.

RESULTS PENDING: per-stage timings (requirements, evidence, risk, deterministic, LLM judge, report) and peak RSS.

Cost and energy. Cost is measured as the equivalent USD reported by the CLI per call (`total_cost_usd`; the subscription itself is a flat fee) together with input/output tokens. We cannot meter the energy of a hosted model directly, so tokens and the number of calls are used as proxies: inference energy grows with generated and processed tokens [17, 30]. Three design choices reduce them: (1) *routing* – the LLM sees only the requirements the rules cannot settle; (2) *bounded prompts* – at most 14 evidence items and 12 000 characters per judgement, one requirement per call, no tool use; (3) a *content-addressed cache*, so a re-audit of an unchanged repository (the common CI case) costs no new calls.

RESULTS PENDING: cold vs. warm cost and calls per audit; tokens per configuration.

6.6 Explainability

Every verdict is explainable by construction: it carries the rule-check trace (which check passed or failed and where), the judge's rationale that refers to evidence aliases, the cited `file:line` snippets, the gaps and the remediation; conflicts keep both the rule and the LLM rationale. `ccauditor explain` and the dashboard side sheet render this justification for a single requirement, and SARIF places the finding on the cited lines in code review.

6.7 Discussion

RESULTS PENDING: answer RQ1–RQ4 with the numbers: is the hybrid hypothesis supported; where does each mode fail (per category, per app); effect of the Verifier and of model size; statistical significance; cost per correct verdict.

Independently of the final numbers, the design implies the following expected failure modes, which the analysis should confirm or refute: (i) the rule engine is precise on decoy-free, syntactic requirements but propagates confident false negatives on indirect evidence (Section 5.12); (ii) the LLM-only mode recovers indirect evidence but tends to be stricter, flagging more items as gaps (higher gap recall, lower gap precision) and costing an order of magnitude more calls; (iii) the hybrid mode inherits the rule engine's confident errors because they are never routed, which bounds its accuracy from above and is the main target for threshold tuning; (iv) data-governance and human-oversight requirements are the hardest for rules, because their evidence is mostly prose.

Threats to validity. The benchmark is small (5 apps, 225 labels), synthetic and labelled by the authors who also wrote the rules, which risks optimistic results for the deterministic mode on the patterns we anticipated; we mitigated this by labelling before running the rules and by planting traps

against our own rules. Policies are selected provisions, requirements are curated, and the AI Act will be complemented by harmonised standards that may change what counts as sufficient evidence.

Chapter 7

Governance aspects

An AI system that audits AI systems must itself be governed. We analyse the auditor along the course's governance dimensions; Table 7.1 summarises its agency.

7.1 Ethics

The main ethical risk is *false assurance*: a report that says "compliant" where the law is not met can harm the people affected by the audited system (rejected candidates, declined borrowers) and shift blame to a tool. We therefore (i) treat a false compliance as the worst error in both the metrics and the design (no positive verdict without a verified citation; disagreement ⇒ human); (ii) state in every report that it is decision support for an internal audit [27], not a conformity assessment or legal advice; (iii) never let generated artefacts invent facts (explicit `[TODO]` placeholders). *Privacy:* the auditor reads source repositories that may contain secrets or personal data in sample logs; only bounded excerpts are sent to the LLM, through the user's own Claude account, with no tool access and no session persistence, and the local cache can be disabled or wiped. A deployment on confidential code should add secret/PII redaction before prompting, which is listed as future work. *Dual use:* the same tool could be used to produce a "clean" report by tuning rules to pass; the full rule trace and the report history make such manipulation visible.

7.2 Fairness

Fairness concerns the auditor in two ways. First, *fairness of the audited systems*: the requirements include data governance and bias examination (AI Act Art. 10), and the benchmark contains a system that uses age, gender and nationality as features; the auditor can check that bias evaluation is *documented and performed*, but it does not measure bias itself (it has no access to data or predictions), and it should not be read as a fairness certification [19]. Second, *fairness of the auditor across the audited*: the same requirement should receive the same verdict for equivalent evidence regardless of programming language, documentation language or naming style. Rules are biased toward the vocabulary they were written for (e.g. English identifiers, `logger`); the LLM is more robust to naming but may favour verbose, well-documented repositories. We report per-category and per-application results to expose such disparities; extending the benchmark to non-English documentation and more stacks is future work.

7.3 Agency: what the system may decide and how far it may act

Following the levels-of-automation view of Parasuraman et al. [26], the auditor automates information acquisition and analysis fully, *recommends* decisions, and *executes* nothing.

Table 7.1: Agency of the auditor: decisions it may take alone, with a human, or never.

Level	Decisions	Condition
Autonomous	collect evidence, classify risk tier, mark NA, emit C/P/NC verdicts, draft remediation and artefacts, fail a CI build on a regression	verdict verified, confidence $\geq \beta$, no rule/LLM conflict; CI failure is reversible and threshold set by the team
Human-in-the-loop	any verdict with confidence $< \beta$, any rule vs. LLM conflict, any C claim without valid citation, contested verdicts	routed to the review queue as NR; a named reviewer decides
Never	modify the audited repository, commit or merge, file anything with a regulator, declare legal conformity, decide applicability by LLM alone	out of scope by design (no write tools; judge cannot output NA)

The system may therefore act *only on its own outputs* (a report, a SARIF file, a CI exit code). The furthest it acts before a person is involved is blocking a merge in CI, which is reversible, configurable (fail-on severity) and based on a regression relative to a baseline that a human accepted.

7.4 Architecture: components required by that permission

Each permission in Table 7.1 is backed by a concrete component:

- **Retrieval**: the evidence index and the BM25 EvidenceScout bound what the judge can see; every claim must be traceable to an indexed chunk with path and line span.
- **Tools**: none for the LLM. The Claude Code CLI runs with tools, MCP servers and settings disabled in an empty temporary directory; only the deterministic code reads files. This removes the risk of an agent modifying the repo.
- **Memory**: no conversational memory between calls (no session persistence); the only memory is the content-addressed response cache (reproducibility) and the stored reports with their review history (institutional memory, input to CI regression checks).
- **Orchestration**: a fixed, ordinary-code pipeline (Algorithm 1) with explicit routing thresholds; agents cannot spawn agents or change the plan.
- **Checks**: JSON-schema validation with bounded retries, the Verifier (Algorithm 2), the conflict rule and the confidence gate, and the CI regression comparator.

7.5 Accountability: who answers, what the record shows, how to contest

Accountability for AI requires an identifiable answerable agent, a record and a forum [23].

- **Who answers.** The organisation deploying the audited system (the AI Act provider or deployer) remains responsible for compliance; within it, the named reviewer who resolves an item answers for that decision, and the maintainers of the requirements YAML answer for the rules. The tool's authors answer for the correctness of the pipeline and its documented limits.
- **What the record shows.** Each report stores the policy ids, the full requirements used, the mode, model and backend, the risk tier with triggers, every verdict with method, confidence, verified flag, rationale, cited evidence (path, lines, snippet), gaps and remediation, plus timings, LLM calls, tokens and cost. Human overrides append "*Human review by X: old → new. Justification*" and are listed in the report statistics (`human_reviews`); the cache makes LLM outputs replayable.
- **How to contest.** Any verdict can be contested in the dashboard review queue or via `ccauditor review` / `POST /reports/review`; the contester sees the exact evidence and both rationales, and the decision is recorded with name and justification, in line with contestability by design [1]. Rule-level errors are contested by changing the versioned requirement YAML, and the policy-diff tool shows which reports are affected by such a change.

Transparency. The rules, prompts, thresholds and benchmark are open in the repository; the report exposes the method that produced each verdict, so a reader can distinguish rule results, LLM judgements and human decisions.

Chapter 8

SWOT analysis

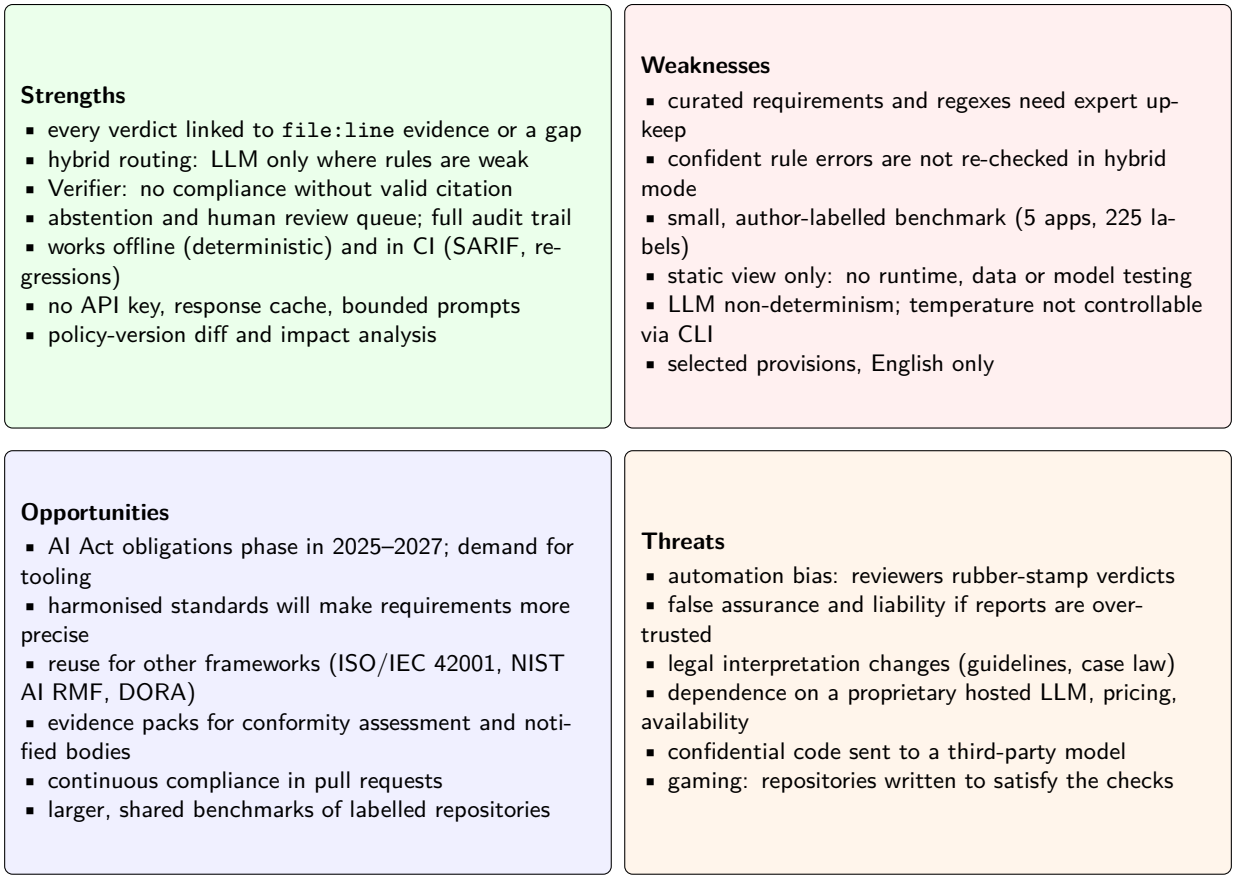


Figure 8.1: SWOT analysis of the Compliance-as-Code AI Auditor.

Mitigations. Automation bias is countered by showing evidence before the verdict in the review sheet and by requiring a justification for every override; vendor dependence by the backend protocol (any model behind `complete_json`) and the deterministic fallback; gaming by the LLM judge, which reads the whole context instead of matching tokens, and by the decoy-style tests of the benchmark.

Chapter 9

Conclusions and future work

We framed the checking of an application repository against a regulation as grounded, selective classification: every requirement receives a status, a confidence and citations, and the system may abstain in favour of a human. The Compliance-as-Code AI Auditor implements this with a hybrid of deterministic policy-as-code checks and an LLM judge (Claude via the Claude Code CLI in headless mode) constrained by retrieval, a strict citation contract, a Verifier and a human-in-the-loop gate. It turns three policies (selected EU AI Act provisions, GDPR, an internal responsible-AI policy) into 45 testable requirements and produces auditable reports in JSON, Markdown, SARIF and a web dashboard, with CI regression gating and policy-version impact analysis.

RESULTS PENDING: two or three sentences with the main quantitative findings (RQ1–RQ4) and the recommended default configuration.

Most important points. (1) Rules and LLMs fail in complementary ways, so routing between them is worth more than either alone; (2) the costliest auditor error, false compliance, can be controlled structurally (citations, conflicts, abstention) rather than only statistically; (3) an auditor must be auditable: the record of who or what decided each verdict, on which evidence, is as important as the verdict.

Future work.

- *Routing*: route confident negative rule verdicts too, or learn the routing policy from review outcomes (active learning from the human queue).
- *Requirements*: evaluate PolicyAnalyst-derived requirements against the curated ones; cover the full AI Act and harmonised standards; multilingual policies and repositories.
- *Evidence*: dense or hybrid retrieval, cross-file reasoning (call graphs), runtime evidence (logs, metrics, model evaluations) in addition to static artefacts.
- *Safety*: secret/PII redaction before prompting, local open-weight models for confidential code, calibration of the judge's confidence on review outcomes.
- *Evaluation*: more and larger repositories labelled by independent annotators, with inter-annotator agreement.

Bibliography

[1] Kars Alfrink, Ianus Keller, Gerd Kortuem, and Neelke Doorn. Contestable AI by design: Towards a framework. *Minds and Machines*, 33(4):613–639, 2022. doi: 10.1007/s11023-022-09611-z.

[2] Orlando Amaral Cejas, Muhammad Ilyas Azeem, Sallam Abualhaija, and Lionel C. Briand. NLP-based automated compliance checking of data processing agreements against GDPR. *IEEE Transactions on Software Engineering*, 49(9):4282–4303, 2023. doi: 10.1109/TSE.2023.3288901. arXiv:2209.09722.

[3] Anthropic. Claude code documentation: headless mode and command-line reference, 2025. URL https://docs.anthropic.com/en/docs/claude-code/overview.

[4] Travis D. Breaux and Annie I. Antón. Analyzing regulatory rules for privacy and security requirements. *IEEE Transactions on Software Engineering*, 34(1):5–20, 2008. doi: 10.1109/TSE.2007.70746.

[5] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Katz, and Nikolaos Aletras. LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4310–4330, 2022. doi: 10.18653/v1/2022.acl-long.297. arXiv:2110.00976.

[6] European Parliament and Council of the European Union. Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (general data protection regulation). Official Journal of the European Union, L 119, 4 May 2016, 2016. URL http://data.europa.eu/eli/reg/2016/679/oj.

[7] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, L series, 12 July 2024, 2024. URL http://data.europa.eu/eli/reg/2024/1689/oj.

[8] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6465–6488, 2023. doi: 10.18653/v1/2023.emnlp-main.398. arXiv:2305.14627.

[9] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. arXiv:1803.09010.

[10] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, et al. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2308.11462. arXiv:2308.11462.

[11] Philipp Hacker, Andreas Engel, and Marco Mauer. Regulating ChatGPT and other large generative AI models. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 1112–1123, 2023. doi: 10.1145/3593013.3594067.

[12] Shabnam Hassani, Mehrdad Sabetzadeh, Daniel Amyot, and Jain Liao. Rethinking legal compliance automation: Opportunities with large language models, 2024. URL https://arxiv.org/abs/2404.14356. arXiv:2404.14356.

[13] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.

[14] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, et al. Language models (mostly) know what they know, 2022. URL https://arxiv.org/abs/2207.05221. arXiv:2207.05221.

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474, 2020. URL https://arxiv.org/abs/2005.11401. arXiv:2005.11401.

[16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. arXiv:2307.03172.

[17] Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power hungry processing: Watts driving the cost of AI deployment? In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 85–99, 2024. doi: 10.1145/3630106.3658542. arXiv:2311.16863.

[18] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9004–9017, 2023. doi: 10.18653/v1/2023.emnlp-main.557. arXiv:2303.08896.

[19] Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan. A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6):1–35, 2021. doi: 10.1145/3457607. arXiv:1908.09635.

[20] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on*

Empirical Methods in Natural Language Processing (EMNLP), pages 12076–12100, 2023. doi: 10.18653/v1/2023.emnlp-main.741. arXiv:2305.14251.

[21] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229, 2019. doi: 10.1145/3287560.3287596. arXiv:1810.03993.

[22] Jakob Mökander, Jonas Schuett, Hannah Rose Kirk, and Luciano Floridi. Auditing large language models: a three-layered approach. *AI and Ethics*, 4(4):1085–1115, 2023. doi: 10.1007/s43681-023-00289-2. arXiv:2302.08500.

[23] Claudio Novelli, Mariarosaria Taddeo, and Luciano Floridi. Accountability in artificial intelligence: what it is and how it works. *AI & Society*, 39(4):1871–1882, 2023. doi: 10.1007/s00146-023-01635-y.

[24] OASIS. Static analysis results interchange format (SARIF) version 2.1.0. OASIS Standard, 27 March 2020, 2020. URL https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html.

[25] Open Policy Agent contributors. Open policy agent: policy-based control for cloud native environments (rego policy language). Cloud Native Computing Foundation graduated project, 2024. URL https://www.openpolicyagent.org/docs/latest/.

[26] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3):286–297, 2000. doi: 10.1109/3468.844354.

[27] Inioluwa Deborah Raji, Andrew Smart, Rebecca N. White, Margaret Mitchell, Timnit Gebru, Ben Hutchinson, Jamila Smith-Loud, Daniel Theron, and Parker Barnes. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 33–44, 2020. doi: 10.1145/3351095.3372873. arXiv:2001.00973.

[28] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 2023. doi: 10.1162/coli_a_00490. arXiv:2112.12870.

[29] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 2009. doi: 10.1561/1500000019.

[30] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3645–3650, 2019. doi: 10.18653/v1/P19-1355. arXiv:1906.02243.

[31] Damiano Torre, Sallam Abualhaija, Mehrdad Sabetzadeh, Lionel Briand, Katrien Baetens, Peter Goes, and Sylvie Forastier. An AI-assisted approach for checking the completeness of privacy

policies against GDPR. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*, pages 136–146, 2020. doi: 10.1109/RE48521.2020.00025.

[32] Michael Veale and Frederik Zuiderveen Borgesius. Demystifying the draft EU artificial intelligence act: Analysing the good, the bad, and the unclear elements of the proposed approach. *Computer Law Review International*, 22(4):97–112, 2021. doi: 10.9785/cri-2021-220402. arXiv:2107.03721.

[33] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155. arXiv:2308.08155.

[34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629. arXiv:2210.03629.

[35] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.05685. arXiv:2306.05685.